UTEC
Universidad Tecnológica

Estructuras de datos

Una estructura de datos es una forma de organizar y almacenar datos de manera que puedan ser accedidos y modificados de manera eficiente.

Tipos de estructuras de datos

- Estructuras de datos primitivas (int, float, char, etc.).
- Estructuras de datos no primitivas (arrays, listas enlazadas, pilas, colas, árboles, grafos, etc.).

Clasificación de estructuras de datos

Estructuras de datos primitivas

Las estructuras de datos primitivas son las más básicas y están integradas directamente en el lenguaje de programación. Incluyen tipos de datos simples y básicos.

- Ejemplos:
  - Enteros (`int`)
  - Flotantes (`float`)
  - Caracteres (`char`)
  - Booleanos (`boolean`)

Estructuras de datos no primitivas

Estas son más complejas y pueden ser construidas utilizando estructuras primitivas. Se dividen en varias categorías según su organización y características.

Estructuras de datos lineales

En las estructuras de datos lineales, los elementos se organizan en una secuencia lineal. Cada elemento tiene un único predecesor y un único sucesor (excepto el primero y el último elemento).

Arrays (Arreglos):
  - Definición: Colección de elementos del mismo tipo almacenados en ubicaciones de memoria contiguas.
  - Usos: Acceso rápido a elementos por índice.

- Listas Enlazadas (Linked Lists):
  - Definición: Colección de nodos donde cada nodo contiene un dato y una referencia (enlace) al siguiente nodo en la secuencia.
  - Tipos:
    - Listas Enlazadas Simples (Singly Linked List)
    - Listas Enlazadas Dobles (Doubly Linked List)
    - Listas Circulares (Circular Linked List)
  - Usos: Inserciones y eliminaciones frecuentes.

- Pilas (Stack):
  - Definición: Colección de elementos con acceso en modo LIFO (Last In, First Out).
  - Operaciones: `push` (inserción), `pop` (eliminación), `peek` (visualización del elemento superior).
  - Usos: Manejo de expresiones aritméticas, deshacer operaciones en editores de texto.

- Colas (Queue):
  - Definición: Colección de elementos con acceso en modo FIFO (First In, First Out).
  - Operaciones: `enqueue` (inserción), `dequeue` (eliminación), `peek` (visualización del elemento frontal).
  - Variantes:
    - Cola Doble (Deque)
    - Cola de Prioridad (Priority Queue)
  - Usos: Gestión de tareas en sistemas operativos, simulaciones de colas de espera.

Estructuras de datos no lineales

En las estructuras de datos no lineales, los elementos no se organizan en una secuencia lineal. Pueden tener múltiples relaciones entre ellos, lo que permite una representación más compleja de los datos.

- Árboles (Trees):
  - Definición: Estructura jerárquica con un nodo raíz y nodos hijos, formando una relación de padres e hijos.
  - Tipos:
    - Árbol Binario
    - Árbol Binario de Búsqueda (Binary Search Tree)
    - Árbol AVL (Árbol Binario de Búsqueda Balanceado)
    - Árbol Rojo-Negro
  - Operaciones: inserción, eliminación, búsqueda.
  - Usos: Representación de jerarquías, sistemas de archivos, bases de datos.

- Grafos (Graphs):

- Definición: Conjunto de nodos (o vértices) y aristas (o enlaces) que conectan pares de nodos.
- Tipos:
  - Dirigidos (Directed Graphs)
  - No Dirigidos (Undirected Graphs)
- Representación:
  - Matriz de Adyacencia
  - Lista de Adyacencia
- Operaciones: recorrido en anchura (BFS), recorrido en profundidad (DFS).
- Usos: Modelado de redes sociales, rutas de transporte, redes de comunicación.

Otras Estructuras de Datos

- Tablas Hash (Hash Tables):
  - Definición: Estructura que asocia claves con valores utilizando una función hash para calcular el índice de almacenamiento.
  - Operaciones: inserción, eliminación, búsqueda.
  - Manejo de Colisiones: encadenamiento, direccionamiento abierto.
  - Usos: Implementación de diccionarios, bases de datos.

Comparación y selección de estructuras de datos

La elección de una estructura de datos depende de varios factores, incluyendo:

- Eficiencia en tiempo:
  - ¿Qué tan rápido se pueden realizar las operaciones de inserción, eliminación y búsqueda?

- Eficiencia en espacio:
  - ¿Cuánta memoria se utiliza?

- Complejidad del problema:
  - ¿Qué tipo de relaciones o jerarquías existen en los datos?

- Frecuencia de operaciones:
  - ¿Cuáles operaciones se realizan con más frecuencia?

Entender y clasificar las estructuras de datos es fundamental para resolver problemas de manera eficiente. Cada estructura tiene sus ventajas y desventajas, y conocerlas te permitirá tomar decisiones informadas sobre cuál utilizar en diferentes contextos y aplicaciones.